



Bitácora de Desempeño Corte 01

Tercer cuatrimestre

Procesamiento de Lenguaje Natural (PLN)

Mtro. Pablo Ricardo Sanchez Gomez
Jessica Melani Romero Lora
Matrícula 253220116

10 de mayo del 2026

Tabla de contenido

1. Introducción del Primer Corte de Evaluación.....	3
2. Construcción de Pipeline inicial de PLN (Clase 01).....	3
1.1 Datos de la práctica.....	3
1.2 Bitácora técnica.....	4
1.3 Salida principal en consola.....	5
1.4 Resultados y observaciones.....	5
1.5 Errores y debugging.....	5
Corrección aplicada:.....	6
1.6 Conclusión.....	6
3. Representación numérica: BoW, TF-IDF y similitud de coseno (Clase 02).....	7
2.1 Datos de la práctica.....	7
2.2 Bitácora técnica.....	7
Clase 2 — Confirmación del vocabulario ampliado (Mini-reto 2).....	9
2.3 Salida principal — Mini-reto 1 (18 documentos).....	10
2.4 Salida — Mini-reto 2 (+3 comentarios, 21 documentos).....	10
2.5 Análisis de resultados.....	10
2.6 Error documentado.....	11
Corrección aplicada:.....	11
2.7 Conclusión.....	11
4. Clasificación Supervisada en Primera Etapa Clase 03.....	12
3.1 Datos de la práctica.....	12
3.2 Bitácora técnica.....	12
3.3 Métricas obtenidas.....	13
3.4 Matriz de confusión — Regresión Logística.....	13
3.5 Predicciones nuevos comentarios.....	14
3.6 Análisis de errores.....	14
Al abrir 06_predicciones_prueba.csv se observa:.....	15
3.7 Conclusiones.....	15
5. Ampliación de Dataset: ejecución de modelos, comparación y análisis Clase 04....	16
4.1 Datos de la práctica.....	16
4.2 Bitácora técnica.....	17
4.3 Mini-reto 1 — Comparativo de modelos (45 documentos).....	17
4.4 Mini-reto 2 — Resultado tras agregar 10 comentarios (55 documentos).....	18
4.5 Defensa del mejor modelo.....	19
4.6 Error lógico documentado.....	20
4.7 Conclusiones.....	21

1. Introducción del Primer Corte de Evaluación

El Corte 1 cubre la Unidad 1 completa de la asignatura: fundamentos de PLN, preprocesamiento y representación de texto, clasificación supervisada inicial y comparación formal de modelos. Las cuatro prácticas de clase establecen el pipeline base que se utilizará como referencia durante el resto del cuatrimestre.

Temas trabajados	Herramientas utilizadas
Pipeline de preprocesamiento (limpieza, tokenización, stopwords)	Python 3.11 + Miniconda + Visual Studio Code
Representación numérica: BoW, TF, TF-IDF, similitud coseno	pandas, csv, math, unicodedata, re (Clase 1-2)
Clasificación supervisada: Naive Bayes y Regresión Logística	scikit-learn: TfidfVectorizer, Pipeline, métricas (Clase 3-4)
Comparación formal de 4 modelos (NB, CNB, LR, SVM)	matplotlib (matrices de confusión en PNG)
Análisis de errores y confusiones lógicas entre categorías	CSV de entrada y carpetas de salida estructuradas

Tabla 01. Presentación de temas trabajados y herramientas utilizadas. Fuente: elaboración propia con los resultados obtenidos del primer corpus

2. Construcción de Pipeline inicial de PLN (Clase 01)

1.1 Datos de la práctica

Campo	Detalle
Nombre de la práctica	Pipeline inicial de PLN
Objetivo técnico	Construir manualmente las etapas de limpieza, tokenización y eliminación de stopwords para procesar texto en español

Archivo principal	pipeline_inicial_clase1.py
Datos de entrada	comentarios_clase1.csv (textos libres en español)
Salidas generadas	01_tokens_y_frecuencias.csv · 02_vocabulario.csv · 03_estadisticas.csv
Entorno	Python 3.11 · Miniconda (pln311) · VS Code

Tabla 02. Presentación de temas trabajados y herramientas utilizadas. Fuente: elaboración propia con los resultados obtenidos del primer corpus

1.2 Bitácora técnica

Ruta de trabajo: masters/PLN/clase1/

Ambiente activo: conda activate pln311

Comando ejecutado: python pipeline_inicial_clase1.py

Dependencias: pathlib, csv, re, unicodedata, collections.Counter (biblioteca estándar, sin instalación adicional)

Pasos ejecutados

- Carga de comentarios_clase1.csv con csv.DictReader (encoding utf-8-sig).
- Normalización: lowercase → quitar_acentos() → re.sub para eliminar caracteres no alfanuméricos.
- Tokenización por split() sobre texto limpio.
- Filtrado de stopwords y tokens de longitud ≤ 2 con comprensión de lista.
- Conteo de frecuencias con collections.Counter y construcción del vocabulario ordenado.
- Exportación de tres archivos CSV en carpeta salidas_clase1/.

1.3 Salida principal en consola

Configuración del ambiente

```
conda activate pln311
cd masters/PLN/clase1/
python pipeline_inicial_clase1.py
```

```
python pipeline_inicial_clase1.py Comentarios procesados: 18
Vocabulario único: 87 términos Token más frecuente: plataforma (6)
Archivos generados en :/home/usuario/PLN/clase1/salidas_clase1
```

1.4 Resultados y observaciones

Resultado clave

La función `quitar_acentos()` permite unificar formas como 'útil' y 'util', evitando duplicados en el vocabulario.

Los tokens de una sola letra y preposiciones se eliminan correctamente mediante la condición `len(token) > 2` combinada con la lista de stopwords.

El vocabulario de 87 términos sobre 18 documentos refleja una densidad léxica alta, típica de comentarios cortos y específicos.

1.5 Errores y debugging

Error documentado

Mensaje: `UnicodeDecodeError` al leer el CSV.

Momento: lectura inicial del archivo con `ruta.open('r', encoding='utf-8')`.

Causa probable: el archivo fue guardado con BOM (Byte Order Mark) por Excel.

Acción: cambiar encoding a 'utf-8-sig'. El parámetro `newline=""` también es necesario para `csv.DictReader`.

Resultado: error resuelto. El archivo se carga correctamente.

```
python pipeline_inicial_clase1.py
Traceback (most recent call last):
  File "pipeline_inicial_clase1.py", line 48, in
cargar_comentarios
    with ruta.open("r", encoding="utf-8") as archivo:
  File "/usr/lib/python3.11/codecs.py", line 322, in decode
    (result, consumed) = self._buffer_decode(data,
self.errors, final)
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xef in
position 0:
invalid continuation byte
```

Corrección aplicada:

```
python pipeline_inicial_clase1.py
Traceback (most recent call last):
  File "pipeline_inicial_clase1.py", line 48, in
cargar_comentarios
    with ruta.open("r", encoding="utf-8-sig") as archivo:
  File "/usr/lib/python3.11/codecs.py", line 322, in decode
    (result, consumed) = self._buffer_decode(data,
self.errors, final)
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xef in
position 0:
invalid continuation byte
```

1.6 Conclusión

El pipeline manual permite entender con precisión cada etapa de preprocesamiento. La separación de funciones (limpiar_texto, tokenizar, quitar_stopwords) facilita la reutilización y el diagnóstico de errores. La mayor limitación detectada es que el filtrado por stopwords es estático: no considera contexto, lo que puede eliminar términos relevantes en frases negativas.

3. Representación numérica: BoW, TF-IDF y similitud de coseno (Clase 02)

2.1 Datos de la práctica

Campo	Detalle
Nombre de la práctica	Representación de texto: BoW, TF, TF-IDF y similitud
Objetivo técnico	Convertir el corpus de comentarios en representaciones numéricas y calcular similitud entre documentos
Archivo principal	pipeline_representacion_clase2.py
Datos de entrada	comentarios_clase2.csv (18 comentarios originales + 3 nuevos — Mini-reto 2)
Salidas generadas	01_preprocesamiento.csv · 02_matriz_bow.csv · 03_matriz_tf.csv · 04_matriz_tfidf.csv · 05_top_terminos_tfidf.csv · 06_similitud_documentos.csv
Entorno	Python 3.11 · Miniconda (pln311) · VS Code

Tabla 03. Presentación de temas trabajados y herramientas utilizadas. Fuente: elaboración propia con los resultados obtenidos del primer corpus

2.2 Bitácora técnica

Ruta de trabajo: masters/PLN/clase2/

Comando ejecutado: python pipeline_representacion_clase2.py

Dependencias: pathlib, csv, math, re, unicodedata, collections.Counter (biblioteca estándar)

Pasos ejecutados

- Carga y preprocesamiento del corpus con las mismas funciones de Clase 1 (reutilización).

- Construcción del vocabulario ordenado con `set()` + `sorted()` sobre todos los tokens útiles.
- Cálculo de la matriz BoW con `collections.Counter` por documento.
- Cálculo de TF: frecuencia relativa dividiendo conteo entre total de tokens del documento.
- Cálculo de IDF con suavizado de Laplace: $\log((N+1)/(df+1)) + 1$.
- Producto $TF \times IDF$ para obtener la matriz TF-IDF.
- Cálculo de similitud coseno entre todos los pares de documentos.
- Mini-reto 2: se agregaron 3 comentarios nuevos al CSV y se re-ejecutó el pipeline completo.

```
cd ~/PLN/clase2/

# Corrección del nombre de archivo (error documentado)
cp comentarios_clase2_err.csv comentarios_clase2.csv

# Ejecución Mini-reto 1 (18 documentos originales)
python pipeline_representacion_clase2.py

# Mini-reto 2: agregar 3 comentarios y re-ejecutar
python pipeline_representacion_clase2.py
```


Clase 2 — Confirmación del vocabulario ampliado (Mini-reto 2)



```
# Antes (18 documentos)
Documentos procesados : 18
Tamaño del vocabulario: 113

# Después de agregar 3 comentarios (21 documentos)
Documentos procesados : 21
Tamaño del vocabulario: 131
```

2.3 Salida principal — Mini-reto 1 (18 documentos)



```
python pipeline_representacion_clase2.py Documentos procesados
: 18 Tamaño de vocabulario: 113 Archivos generados en :
masters/PLN/clase2/salidas_clase2 Top 5 similitudes:
doc_a=4, doc_b=17 → 0.1248   doc_a=8, doc_b=13 → 0.1248
doc_a=9, doc_b=17 → 0.1248
```

2.4 Salida — Mini-reto 2 (+3 comentarios, 21 documentos)



```
python pipeline_representacion_clase2.py Documentos procesados
: 21 Tamaño de vocabulario: 131 (+18 términos nuevos) Top
similitud: doc_a=5, doc_b=19 → 0.1324 (nuevo par)
```

2.5 Análisis de resultados

Términos relevantes: causas de su importancia en los resultados

TF-IDF premia términos frecuentes en un documento pero raros en el corpus.

Cada comentario usa vocabulario único → IDF alto → términos específicos dominan.

Similitudes coseno bajas (< 0.15) indican corpus léxicamente diverso.

Agregar comentarios nuevos amplió el vocabulario de 113 a 131 términos.

Cambios al agregar 3 comentarios nuevos

El par doc_5–doc_19 compartió términos de 'conceptos confusos' → similitud máxima.

Los 3 comentarios nuevos introdujeron tokens únicos (coseno, corpus, ruido).

IDF de términos existentes bajó ligeramente al aumentar la frecuencia documental.

Las similitudes anteriores se redujeron levemente por el ajuste del IDF global.

2.6 Error documentado

Errores identificados durante la práctica

Nombre del archivo de entrada con sufijo '_err' (comentarios_clase2_err.csv).

Momento: al ejecutar, el script lanzó FileNotFoundError porque esperaba comentarios_clase2.csv.

Causa: nombre de archivo incorrecto en ARCHIVO_ENTRADA.

Acción: copiar el archivo con el nombre correcto antes de ejecutar el script.

Resultado: resuelto. El script ejecuta correctamente con el nombre esperado.



```
$ python pipeline_representacion_clase2.py
Traceback (most recent call last):
  File "pipeline_representacion_clase2.py", line 134, in main
    comentarios = cargar_comentarios(ARCHIVO_ENTRADA)
  File "pipeline_representacion_clase2.py", line 57, in
cargar_comentarios
    raise FileNotFoundError(f"No existe el archivo de entrada:
{ruta.resolve()}")
FileNotFoundError: No existe el archivo de entrada:
/home/usuario/PLN/clase2/comentarios_clase2.csv|
```

Corrección aplicada:



```
cp comentarios_clase2_err.csv comentarios_clase2.csv
python pipeline_representacion_clase2.py
```

2.7 Conclusión

La implementación manual de TF-IDF permite comprender la mecánica de ponderación que scikit-learn abstrae. La similitud coseno revela pares

temáticamente relacionados sin necesidad de etiquetas. El resultado más importante es que agregar datos mejora la representatividad del vocabulario, aunque las similitudes entre documentos cortos y específicos serán siempre bajas dada la dispersión léxica natural del corpus.

4. Clasificación Supervisada en Primera Etapa Clase 03

3.1 Datos de la práctica

Campo	Detalle
Nombre de la práctica	Clasificación supervisada: Naive Bayes y Regresión Logística
Objetivo técnico	Entrenar y evaluar dos modelos supervisados para clasificar comentarios en tres categorías
Archivo principal	pipeline_clasificacion_clase3.py
Datos de entrada	comentarios_entrenamiento_clase3.csv (45 filas, 3 categorías) · comentarios_nuevos_clase3.csv (5 filas)
Salidas generadas	01_datos_preprocesados.csv · 02_metricas_modelos.csv · 03_vocabulario_tfidf.csv · 04_matriz_tfidf.csv · 05_matriz_confusion_mejor_modelo.csv · 06_predicciones_prueba.csv · 07_predicciones_nuevos_comentarios.csv
Entorno	Python 3.11 · Miniconda (pln311) · scikit-learn · pandas · VS Code

Tabla 04. Presentación de temas trabajados y herramientas utilizadas. Fuente: elaboración propia con los resultados obtenidos del primer corpus

3.2 Bitácora técnica

Ruta de trabajo: masters/PLN/clase3/

Instalación: pip install pandas scikit-learn

Comando ejecutado: python pipeline_clasificacion_clase3.py

Pipeline implementado

- TfidfVectorizer con preprocessor=limpiar_texto, stop_words, ngram_range=(1,2), max_features=120, sublinear_tf=True.
- Pipeline de scikit-learn: etapa 'tfidf' + etapa 'modelo' en flujo reproducible.
- División train/test: 70%/30%, random_state=42, estratificada por categoría.
- Evaluación con accuracy_score, precision_recall_fscore_support (macro) y classification_report.
- Exportación de vocabulario, matriz TF-IDF, predicciones y confusión en CSV.

```
cd masters/PLN/clase3/

# Instalación de dependencias
pip install pandas scikit-learn

# Ejecución
python pipeline_clasificacion_clase3.py
```

3.3 Métricas obtenidas

Modelo	Accuracy	Precisión	Recall	F1 Macro
Regresión Logística ★	0.786	0.838	0.800	0.765
Naive Bayes Multinomial	0.714	0.711	0.733	0.705

Tabla 05. Métricas obtenidas. Fuente: elaboración propia con los resultados obtenidos del primer corpus

3.4 Matriz de confusión — Regresión Logística

Real \ Predicho	Contenido	Docente	Plataforma
contenido	4	0	0
docente	0	5	0

plataforma	1	2	2
------------	---	---	---

Tabla 06. Elaboración de matriz de confusión. Fuente: elaboración propia con los resultados obtenidos del primer corpus

3.5 Predicciones nuevos comentarios

ID	Texto (resumido)	Predicción
101	El profesor explica bien pero la plataforma falló...	docente △
102	Necesitamos más ejemplos de clasificación...	contenido ✓
103	El sistema no permitió subir el archivo final	plataforma ✓
104	La docente resolvió dudas del modelo y del código	docente ✓
105	El contenido fue útil aunque faltó tiempo...	contenido ✓

Tabla 07. Posibles nuevos comentarios. Fuente: elaboración propia con los resultados obtenidos del primer corpus

3.6 Análisis de errores

Clase más confundida: plataforma

3 de 5 documentos de 'plataforma' en el conjunto de prueba fueron mal clasificados.

Error lógico (doc 101): 'El profesor explica bien pero la plataforma falló al entregar' → predicho como 'docente'.

Causa: el término 'profesor' tiene mayor peso TF-IDF en el corpus de entrenamiento que 'plataforma'.

Este es un caso real de ambigüedad de categoría — el comentario pertenece semánticamente a ambas clases.

La Regresión Logística superó a Naive Bayes porque pondera features continuas TF-IDF de forma más precisa con corpus pequeños.



```
$ python pipeline_clasificacion_clase3.py
```

Clase 3 - PLN: clasificacion supervisada de textos

Modelo seleccionado: regresion_logistica

Metricas principales:

	modelo	accuracy	precision_macro	recall_macro
f1_macro				
regresion_logistica	0.785714		0.838095	0.800000
	0.764550			

Archivos generados en: .../salidas_clase3



```
$ python pipeline_clasificacion_clase3.py
```

	precision	recall	f1-score	support
contenido	1.00	1.00	1.00	4
docente	0.71	1.00	0.83	5
plataforma	1.00	0.40	0.57	5
macro avg	0.90	0.80	0.80	14
weighted avg	0.90	0.79	0.79	14

Al abrir **06_predicciones_prueba.csv** se observa:



```
texto
categoria_real categoria_predicha
El profesor explica bien pero la plataforma... plataforma
docente ← ERROR
```

3.7 Conclusiones

Con 45 documentos, la Regresión Logística supera a Naive Bayes por 6 puntos de F1. El modelo aprende correctamente categorías con vocabulario diferenciado (docente, contenido) pero falla en plataforma cuando el texto mezcla referencias a otros ámbitos. La limitación principal es el tamaño del corpus: con pocos documentos por clase, una sola predicción incorrecta impacta significativamente las métricas macro.

5. Ampliación de Dataset: ejecución de modelos, comparación y análisis

Clase 04

4.1 Datos de la práctica

Campo	Detalle
Nombre de la práctica	Comparación formal de modelos y análisis de errores con dataset ampliado
Objetivo técnico	Comparar NB, CNB, LR y SVM; analizar errores de forma sistemática; preparar script compatible con datasets tipo Kaggle
Archivo principal	clasificacion_texto_kaggle_clase4.py
Datos de entrada	comentarios_ampliados_clase4.csv (45 → 55 filas tras Mini-reto 2)
Salidas generadas	01_datos_preprocesados.csv · 03_comparativo_modelos.csv · 04_errores_modelos.csv · 05_predicciones_mejor_modelo.csv · matriz_confusion_[modelo].csv + .png (×4)
Entorno	Python 3.11 · scikit-learn · pandas · matplotlib · VS Code
Comando base	python clasificacion_texto_kaggle_clase4.py --csv datos/comentarios_ampliados_clase4.cs


```
v --text-col texto --label-col categoría
--output-dir salidas_clase4
```

Tabla 08. Datos principales de la práctica Fuente: elaboración propia con los resultados obtenidos del primer corpus

4.2 Bitácora técnica

Ruta de trabajo: ~/PLN/clase4/

Instalación: pip install pandas scikit-learn matplotlib

Componentes del script

- Argumentos CLI con argparse: --csv, --text-col, --label-col, --output-dir, --max-rows para compatibilidad con Kaggle.
- TfidfVectorizer: ngram_range=(1,2), min_df=2, max_df=0.90, max_features=5000, sublinear_tf=True.
- Cuatro modelos: MultinomialNB, ComplementNB, LogisticRegression(class_weight='balanced'), LinearSVC(class_weight='balanced').
- Exportación de matriz de confusión en PNG con matplotlib (heatmap azul con valores en celdas).
- Archivo 04_errores_modelos.csv con todos los casos incorrectos de todos los modelos.

```
python clasificacion_texto_kaggle_clase4.py ...

Entrenando: naive_bayes_multinomial... F1=0.6406
Entrenando: complement_naive_bayes... F1=0.6406
Entrenando: regresion_logistica... F1=0.6406
Entrenando: linear_svm... F1=0.7146

Mejor modelo      : linear_svm (F1=0.7146)
Registros totales : 55
Train / Test      : 41 / 14
Clases           : ['contenido', 'docente', 'plataforma']
```

4.3 Mini-reto 1 — Comparativo de modelos (45 documentos)

Modelo	Accuracy	Precisión	Recall	F1 Macro
--------	----------	-----------	--------	----------

Linear SVM ★	0.583	0.583	0.583	0.583
Naive Bayes Multinomial	0.583	0.567	0.583	0.556
Complement Naive Bayes	0.583	0.567	0.583	0.556
Regresión Logística	0.583	0.567	0.583	0.556

Tabla 09. Mini-reto 1: Comparación de modelos. Fuente: elaboración propia con los resultados obtenidos del primer corpus

```

cd mel/PLN/clase4/

# Instalación de dependencias
pip install pandas scikit-learn matplotlib

# Mini-reto 1 (45 documentos)
python clasificacion_texto_kaggle_clase4.py \
    --csv datos/comentarios_ampliados_clase4.csv \
    --text-col texto \
    --label-col categoria \
    --output-dir salidas_clase4

# Mini-reto 2: después de agregar 10 comentarios nuevos al CSV
python clasificacion_texto_kaggle_clase4.py \
    --csv datos/comentarios_ampliados_clase4.csv \
    --text-col texto \
    --label-col categoria \
    --output-dir salidas_clase4

# Uso con dataset tipo Kaggle (comando genérico)
python clasificacion_texto_kaggle_clase4.py \
    --csv datos/archivo_kaggle.csv \
    --text-col News \
    --label-col Type \
    --output-dir salidas_kaggle \
    --max-rows 5000

```

4.4 Mini-reto 2 — Resultado tras agregar 10 comentarios (55 documentos)

Modelo	F1 Macro — 45 docs	F1 Macro — 55 docs	Cambio
--------	--------------------	--------------------	--------

Linear SVM ★	0.583	0.715	+13.2 pp
Naive Bayes Multinomial	0.556	0.641	+8.5 pp
Complement Naive Bayes	0.556	0.641	+8.5 pp
Regresión Logística	0.556	0.641	+8.5 pp

Tabla 10. Mini reto 02. Resultados post nuevos comentarios. Fuente: elaboración propia con los resultados obtenidos del primer corpus

```

# Mini-reto 1 (45 documentos)
linear_svm... F1=0.5833

# Mini-reto 2 (55 documentos)
linear_svm... F1=0.7146

```

Comentarios agregados (3 de 10)

Casos ambiguos introducidos en Mini-reto 2

ID 53: 'El profesor subió el material tarde y la plataforma no lo mostraba bien' → etiqueta: docente (mezcla docente + plataforma).

ID 54: 'La práctica fue clara pero classroom no permitía ver el archivo de salida' → etiqueta: contenido (mezcla contenido + plataforma).

ID 55: 'La retroalimentación del docente llegó por classroom con varios días de retraso' → etiqueta: docente (mezcla docente + plataforma).

Efecto: los modelos clasifican estos textos según el término de mayor peso TF-IDF, no por intención semántica completa.

4.5 Defensa del mejor modelo

¿Por qué Linear SVM es el mejor modelo para este corpus?

Linear SVM supera a los otros 3 modelos por ~7 puntos de F1 macro con 55 documentos.

Con TF-IDF de alta dimensión y clases balanceadas, SVM encuentra la frontera de decisión de máximo margen.

El parámetro `class_weight='balanced'` compensa la ligera diferencia de tamaño entre categorías (19/19/17).

A diferencia de Naive Bayes, SVM no asume independencia entre términos, lo que beneficia en textos con bigramas relevantes.

Clase con más confusiones: 'plataforma' — sus comentarios comparten vocabulario con 'docente' y 'contenido'.

4.6 Error lógico documentado

Caso de error lógico explicado

Texto: 'El profesor no revisó a tiempo la evidencia enviada en classroom'

Categoría real: docente → Categoría predicha: plataforma (todos los modelos)

Análisis: los bigramas 'evidencia enviada' y 'en classroom' tienen IDF muy alto en el corpus de plataforma.

El modelo prioriza el contexto de 'classroom' y 'evidencia' sobre el sujeto 'profesor'.

Clasificación razonable: el error es lógico, no absurdo. El texto mezcla ambos dominios.

Mejora propuesta: etiquetar comentarios mixtos con la categoría principal explícita o usar clasificación multi-etiqueta.

```
python clasificacion_texto_kaggle_clase4.py \
  --csv datos/comentarios_ampliados_clase4.csv \
  --text-col texto --label-col categoria \
  --output-dir salidas_clase4
Traceback (most recent call last):
  File "clasificacion_texto_kaggle_clase4.py", line 9, in
<module>
    import matplotlib
ModuleNotFoundError: No module named 'matplotlib'
```

Corrección aplicada:

```
pip install matplotlib
python clasificacion_texto_kaggle_clase4.py \
    --csv datos/comentarios_ampliados_clase4.csv \
    --text-col texto --label-col categoria \
    --output-dir salidas_clase4
```

4.7 Conclusiones

La Clase 4 demuestra que más datos mejoran consistentemente todos los modelos, y que Linear SVM es la opción más robusta para clasificación de texto con TF-IDF en corpus de tamaño mediano. El análisis sistemático de errores revela que la mayor fuente de confusión no es un fallo del algoritmo, sino **ambigüedad genuina en el etiquetado del corpus**. El script con `argparse` permite reutilizar el mismo pipeline con datasets de Kaggle simplemente cambiando los nombres de columnas.